

Runtime Execution and State

Execution lifecycle, state transitions, retries, HITL, and resume.

Eurskem AI · Agentic Workflow Platform

Generated 2026-08-24

Code revision 8bd16ca4a2a3

Derived from the live repository

Source of truth. Inventories and symbol tables are generated from the current checkout. Narrative explains observed structures and does not replace executable contracts.

Execution lifecycle

Run creation establishes owner scope and a durable record. The compiler constructs graph execution. Every node receives resolved configuration and shared services, returns schema-validated output, emits progress, and records status. Failures are classified and persisted. Human gates create resumable state; resume tokens and checkpoints constrain replay.

State classes

- Workflow definition: versioned authoring contract.
- Run document: durable status, inputs, outputs, errors, and timing.
- Graph checkpoint: resumable execution position.
- Event stream: bounded live/replay progress, not canonical history.
- Conversation state: user-facing turns layered over workflow runs.

File	Responsibility
<code>app/runtime/__init__.py</code>	Phase 4 runtime: YAML to LangGraph compiler and executor.
<code>app/runtime/autofix.py</code>	Auto-fix preflight errors.
<code>app/runtime/compiler.py</code>	Compile a WorkflowSpec into a runnable LangGraph StateGraph.
<code>app/runtime/coordination.py</code>	Small Redis coordination primitives shared by Uvicorn workers.
<code>app/runtime/domain_state.py</code>	Namespaced state for optional use-case packs.
<code>app/runtime/events.py</code>	Bounded, session-scoped pub/sub for workflow Server-Sent Events.
<code>app/runtime/executor.py</code>	Compile a WorkflowSpec and run it.
<code>app/runtime/field_schema.py</code>	Visual field schema — the authoring format behind the Builder's structured output builder.
<code>app/runtime/hitl.py</code>	HITL resume path.
<code>app/runtime/loader.py</code>	Loader module.
<code>app/runtime/logic_preflight.py</code>	Zero-token validation of authored business logic.
<code>app/runtime/node_events.py</code>	Helpers for live event emission in the workflow runtime.
<code>app/runtime/preflight.py</code>	Zero-token workflow validation.
<code>app/runtime/rules.py</code>	Deterministic business-rule evaluation.
<code>app/runtime/schema.py</code>	Schema module.
<code>app/runtime/sleep_guard.py</code>	macOS-only guard against idle sleep while a workflow is executing.
<code>app/runtime/snippet_client.py</code>	App-side client for the snippet-runner sidecar (<code>app/runtime/snippet_daemon.py</code>).
<code>app/runtime/snippet_daemon.py</code>	The snippet-runner sidecar's own entrypoint.
<code>app/runtime/snippet_protocol.py</code>	Shared shape between the app and the snippet-runner sidecar.
<code>app/runtime/state.py</code>	Shared mutable state that flows through every LangGraph node.

File	Responsibility
<code>app/runtime/templating.py</code>	Resolve <code>{{node_id.field.subfield}}</code> expressions inside node configs.
<code>app/workflow/builder_store.py</code>	Durable, file-backed state used by the Workflow Builder.
<code>app/workflow/orchestration.py</code>	Shared execution bookkeeping for a single workflow run.
<code>app/workflow/run_chat_store.py</code>	Durable storage for "Ask AI about this run" conversations.
<code>app/workflow/run_history.py</code>	Durable workflow-run history.
<code>app/workflow/subprocess_callback.py</code>	Deliver a finished child run's result to its waiting Subprocess parent.